

# LLM GPT-3.5 study for sentiment analysis across Utkarsh server, Ohio supercomputer, Google Colab and PC

Lavanya B N<sup>a</sup>, Anitha Rathnam K V<sup>a</sup>, Abhishek Appaji<sup>b,c,\*</sup>, Kiran K<sup>a</sup>, P. Deepa Shenoy<sup>a</sup>, Venugopal K R<sup>a</sup>

<sup>a</sup> Department of Computer Science and Engineering, University Visvesvaraya College of Engineering, Bangalore University, Bangalore, India

<sup>b</sup> Department of Medical Electronics Engineering, B.M.S. College of Engineering, Bangalore, India

<sup>c</sup> Maastricht University, University Eye Clinic Maastricht, Maastricht, the Netherlands

## ARTICLE INFO

### Keywords:

Bidirectional long short-term memory (BiLSTM)  
Convolutional neural networks (CNN)  
Generative pretrained transformer (GPT)  
Gated recurrent unit (GRU)  
Natural language processing (NLP)  
Large language model (LLM)  
Recurrent neural network (RNN)

## ABSTRACT

The major objective of the present research is to inspect sentiment analysis models that have been trained on Twitter corpus by utilising the Large Language Model (LLM) gpt-3-5-turbo-16k version of the Generative Pre-trained Transformer (GPT 3.5) model. Such trained models include the Bidirectional long short-term memory neural network (BiLSTM), Convolutional Neural Networks, Gated Recurrent Unit and Recurrent Neural Network which were used to perform computational tasks on the Ohio Supercomputer and Utkarsh Server, in comparison to work conducted on Google Colab and a Personal Computer (PC). This research also looks at the performance as well as the computational aspects of these models in terms of accuracy, recall, F1-score, time/memory complexity and resource requirements (CPU/GPU) throughout the training and testing phases of each model. The training accuracies in this case were concentrated between 49.91 % and 99.98 % while those of the testing accuracies of the models accounts for about 50.00–75.00 %. For instance, models including BiLSTM and RNN usually exhibit more time complexity because of the nature of the models (sequential computation), on the contrary, CNNs are less time-consuming and are more effective in terms of storage modifying layered architecture. The use of supercomputers and specialized servers reduces training time, but resource constraints on platforms such as personal computers or Colab cause considerable divergence.

## 1. Introduction

BUSINESSES are increasingly using social media platforms in the modern digital environment for customer service and communication. The importance of studying social media competition is highlighted by [1], a thorough study that applies text mining techniques to examine unstructured text material on various platforms. It also highlights the potential of text mining to derive commercial value from huge social media data. Parallel to this, a substantial amount of studies on sentiment analysis has been conducted in an effort to create systems that can recognize and categorize the emotions transmitted in digital texts as a result of the explosion of opinion-rich web resources, according to the survey [2]. Decrypting the emotions expressed in text data is crucial, as highlighted by [3], and sentiment analysis one of the applications of Natural language Processing (NLP) [4] is an essential tool in this regard. The key findings of the study and its ability to estimate customer preferences would be beneficial to economic and marketing research.

Sentiment analysis, or opinion mining, has become an essential technique for identifying and understanding the emotions expressed in text.

Deep learning models are very useful due to their ability to capture complex patterns and details of the information contained in the data. This model uses neural network architectures in research [5], to identify and extract important content in the text, performing clear inference and classification. Deep learning models excel at conceptual analysis tasks by recognizing complex patterns and nuances of context. Using these models allows companies to track and understand what their customers think about their products or services. Companies can analyze insights in customer reviews, media posts, and data to identify areas for improvement, better understand customer satisfaction, and improve their products. Similarly, researchers use emotional intelligence to assess public opinion on various topics, focus on relationships, and determine how people feel about a time or issue.

Large-scale sentiment analysis has benefited from the recent developments in deep learning architectures like Bidirectional Long Short-

\* Corresponding author.

E-mail addresses: [lavanya\\_hulkoti@yahoo.com](mailto:lavanya_hulkoti@yahoo.com) (L. B N), [abhishek.ml@bmsce.ac.in](mailto:abhishek.ml@bmsce.ac.in), [am.appaji@maastrichtuniversity.nl](mailto:am.appaji@maastrichtuniversity.nl) (A. Appaji).

<https://doi.org/10.1016/j.rineng.2024.103218>

Received 23 August 2024; Received in revised form 19 October 2024; Accepted 21 October 2024

Available online 9 November 2024

2590-1230/© 2024 The Author(s). Published by Elsevier B.V. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

Term Memory (BiLSTM), Convolution Neural Networks (CNN), Gated Recurrent Unit (GRU) and Recurrent neural network (RNN). In study [6], they are aimed at achieving better fine grained sentiment polarity. The core of the model lies in converting text components into continuous vector representations. However, there are some research gaps. Firstly, while these models have known applications in sentiment analysis, there exist almost no comparative analyses of the performance of these models on various social media and other datasets. Secondly, computational aspects of these models specifically memory consumption, training period as well as the scalability issues regarding different computing surroundings (personal computers and high performance computing cluster) have not been sufficiently addressed. This gap creates a level of indecision about how businesses can operate in practice when the resources available are in most cases limited by decision making on the models to use in deployment.

The introduction of models like GPT-3.5 based on transformers has changed the whole picture of NLP as we know it. These models have shown the ability to excel in a wide range of tasks and display high performance in understanding the context and fluency of text generation. Surprisingly, the operational efficiency of these models in fine-grained sentiment analysis compared to some traditional deep learning models including Bi-LSTM and CNN has not been fully investigated. This is quite another major research and practical gap since most companies are becoming more and more interested in such models which provide the correct prediction quality while consuming as few resources as possible. To tackle these issues is the aim of this study, which seeks to test and evaluate various deep learning models – Bi-LSTM, CNN, GRU, RNN and GPT-3.5 – on sentiment classification task. Based on large Twitter datasets [6], this study analyzes the differences in the performances of these models in terms of metrics such as accuracy, precision, recall, F1 score, and resource constraints on computation (memory, CPU/GPU consumption, and time). The results are intended to suggest to the companies which models may be recommended for use in the analysis of public opinion in large volumes and what efficacy to resource consumption ratio can be achieved with various models on different resources.

This paper puts to the test various sentiment analysis models on various different computing platforms, such as the Ohio Supercomputer, Utkarsh Servers, Google Colab, and PCs, making sure every platform has been assessed adequately in terms of model performance. Performance metrics such as training and testing accuracies, precision, recall, F1 score, and so forth etc. are used in analyzing predictive performance accurately. Moreover, the study also focuses on parameters such as memory usage, Time Complexity, CPU/GPU usage during the training as well as the testing phases. These information suggests that a lot of attention should be paid by those implementing models on sentiment analysis systems to some new emerging parameters on how resources are deployed for effective outcome in implementation of sentiment analysis models on various platforms.

## 2. Related work

In this research, several studies are discussed that delve into sentiment analysis using various deep learning techniques. Each study focuses on analyzing sentiment from different perspectives, employing diverse datasets and models. The research [6], explores sentiment analysis using deep learning techniques, leveraging bidirectional long short-term memory (BiLSTM) and gated recurrent unit (GRU) models, alongside BERT and RoBERTa word embedding techniques. It utilizes datasets from Twitter and IMDB movie reviews to classify sentiments as positive or negative. The journal [8], introduces an ensemble hybrid deep learning approach for sentiment analysis tasks. The model combines Robustly Optimized Bidirectional Encoder Representations from Transformers (RoBERTa) with Long Short-Term Memory (LSTM), Bidirectional LSTM (BiLSTM), and Gated Recurrent Unit (GRU). The ensemble model achieves remarkable accuracies of 94.9 %, 91.77 %, and 89.81 % on IMDB, Twitter US Airline Sentiment dataset, and Sentiment140 dataset, respectively.

The work [9], emphasizes sentiment classification for tweets using deep learning techniques, particularly Recurrent Neural Networks (RNN) and LSTM. The LSTM model demonstrates high accuracy rates, achieving 88.47 % accuracy for positive/negative classification and 89.13 % and 91.3 % accuracy for positive and negative subclasses, respectively.

The journal [10], evaluates sentiment analysis using a Recurrent Neural Network (RNN) with Word2vec embeddings. It achieves an outstanding accuracy rate of approximately 91.9 % in classifying positive and negative sentiments in Indonesian text. The thesis [11], explores sentiment analysis for customer reviews using deep learning techniques, with the BERT model emerging as the most effective, achieving an accuracy of 73 % and an F1 score of 83 %. The study [12], focuses on sentiment analysis of COVID-19 tweets using deep learning and lexicon-based approaches. The BiLSTM and GRU models achieve high accuracy rates of 92.70 % and 91.24 % for COVID-19 dataset and 92.48 % and 93.03 % for vaccination tweets classification, respectively. The research [13], highlights the importance of sentiment analysis using LSTM-based models, achieving an impressive accuracy of 82.8 % in predicting sentiment from reviews. The paper [14], addresses sentiment analysis using machine learning and deep learning models on movie reviews. The LSTM model with BERT embeddings achieves the highest accuracy of 93 %. The study [15], explores sentiment analysis using a deep learning approach, particularly LSTM with attentive layers, achieving an accuracy of 93.66 %.

Research [16], focuses on sentiment analysis using deep learning models, specifically LSTM and C-LSTM, for multiclass sentiment analysis of textual data. Parameter tuning using Dropout regularization is performed to enhance model accuracy. The work [17], provides a comprehensive review of deep learning techniques in sentiment analysis, focusing on CNN and RNN with LSTM. The survey discusses their strengths, limitations, and performance parameters. This survey [18], investigates recent architectures in sentiment analysis, particularly Transformer-based language models, and discusses their performance on benchmark datasets. This paper [19], investigates sentiment analysis in the Software Engineering community, utilizing machine learning models and deep learning classifiers. The study emphasizes feature selection, class imbalance, and model optimization, achieving significant accuracy in predicting sentiments from textual comments. The work [20], addresses sentiment analysis in social networks, employing word embedding methods and Latent Semantic Analysis. The study demonstrates a remarkable success rate of 93 % in emotion classification.

Recent research has focused on analyzing social media content, sentiment, and detecting aggression and emotion in text data. The study is [21], proposed a data balancing technique using Generative Pre-trained Transformer (GPT-2) to classify aggressive posts/tweets in Hindi-English code-mixed text, achieving improved classification performance. The research in [22], compared traditional machine learning models with GPT-based models for sentiment analysis, indicating GPTs' superior performance. The author of [23], emphasized the limitations of keyword-based approaches in financial sentiment analysis, favoring context-aware models. The study is [24], discussed challenges in sentiment analysis and the role of pre-trained deep learning models. The research in [25], compared large language models with specialized sentiment analysis systems [26], highlighting the adaptability of LLMs in diverse natural language processing tasks.

Generally, these studies indeed indicated that deep learning has been found to play a crucial role in sentiment analysis as it consistently yields high levels of accuracy; moreover, it also suggests the viability of combining various techniques in DL for sentiment analysis across various datasets or fields of application. The current research on GPT 3.5 LLM is a demonstration of its significance in the context of sentiment analysis. According to the choice of word embedding method, overall performance can be greatly affected with accuracy ranging from 49.91 % to 99.98 % when incorporating deep learning models with LLM.

### 3. Methodology

The main objective of this study is to comprehensively evaluate different deep learning models for sentiment analysis in various computing platforms, using a rich dataset along with LLM. The development stages of the model are as shown in Fig. 1. This entails combining appropriate data collection techniques, preprocessing, word embedding strategies, while choosing the architecture of a deep learning model and through intensive training and evaluation on Twitter dataset. In addition, these steps include monitoring resource usage and analyzing time complexity. The next section discusses different stages of model development:

#### 3.1. Experimental setup

This model is programmed in Python which is executed on a Utkarsh Server, Ohio Supercomputer, personal computer using Jupyter Notebook and Google Colab. The environment has been loaded with several Python libraries for enabling deep learning models to be run. Among the installed libraries were TensorFlow, Keras, PyTorch and their dependencies. Furthermore, other important data management and manipulation libraries like pandas, NumPy, scikit-learn have been set up for effective data handling and preprocessing. These libraries combined, give the environment the required capabilities to execute deep learning models without hitches.

#### 3.2. Dataset

The Twitter dataset from the Kaggle [7], provides a big corpus of tweets which is useful for determining polarity. The dataset contains 1.6 million tweets with two attributes: results on the opinions and emotions found in the text in Table 1. Using lambda function maps sentiment attribute '4' a positive to 1 and '0' a negative to 0. Fig. 2 shows the positive and negative ones with the ratio being one-to-one. The work was initially performed on Google Colab and a personal computer, which handled 100,000 records efficiently. However, as the training data size increased, the system crashed due to limited computational power and memory. To overcome these limitations, the model was trained and tested on 1.6 million records using supercomputing machines such as the Ohio Supercomputer and the Utkarsh Server, which provided access to larger memory and greater computational resources.

#### 3.3. Data preprocessing

Initially, we start with fine-grained data preparation such as loading dataset and preprocessing the dataset. In Python the "preprocess\_text" function operates on raw text by doing some series of operations including converting them in lower case, eliminating non-alphabetic characters, tokenizing, lemmatizing and removing stopwords. By doing this they ensure that only the relevant information is used to improve the accuracy of NLP models while filtering out distractions and

**Table 1**

Twitter Sentiment140 SAMPLE.

| Sl no. | Text                                                                                               |
|--------|----------------------------------------------------------------------------------------------------|
| 1      | I am soaked. This is not pleasant                                                                  |
| 2      | Good morning people of twitter. TGIFriday! Thats wassssup! Today is going to be a hellofagood day! |

irrelevant information. After that exploratory data analysis is done to have a better understanding on how sentiment classes are distributed and what features might be good for the text data.

#### 3.4. Generative pretrained transformer (GPT 3.5)

Sentiment analysis tasks are facilitated by the large language model (LLM), which maps words or textual units onto continuous vector representations. In other words, this research benefits from passing pre-processed tweets through Generative Pretrained Transformer GPT-3.5-turbo-16k which is an example of GPT-3.5 model trained by Xenova AI company. Textual data processing for DL models was done with GPT tokenizer from Hugging Face that specifically designed for this model type to convert raw texts into numerical tokens, padding and truncation techniques provide a uniform approach to standardizing input sequences, allowing them to fit into pre-determined length. The following methods are:

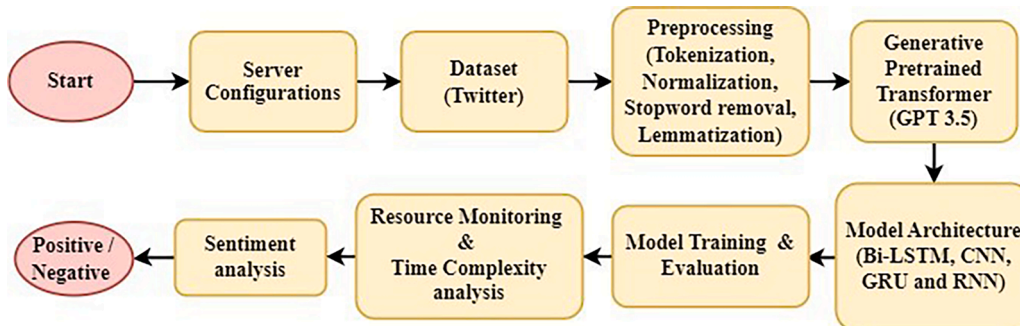
- Importantly, we need to set up tokenizer with the right model and also convert text to numerical tokens that involve padding such as [PAD].
- During padding and truncation sequence must be standardized in relation to max\_length.

#### 3.5. Model architecture

Four different deep learning architectures have been strategically chosen for sentiment analysis: Bidirectional Long Short-Term Memory (Bi-LSTM), Convolutional Neural Network (CNN), Gated Recurrent Unit (GRU) and Recurrent Neural Network (RNN). Each of these architectures comes with its own specific benefits in terms of capturing the dependencies between individual words as well as extracting significant features from textual data thereby making it relevant for various models.

##### I. LSTM Model Architecture

The architecture in Table 2 consists of an Embedding layer, Bidirectional layer and Dense layer. The input sequences after going through the Embedding layer is converted into dense vectors of size 64 per token. For instance, the Bidirectional layer processes inputs bidirectionally giving output shape of 256 units which take advantage of past and future contexts. The dense layer produces a single output prediction. The model has trainable parameters, enabling it to learn complex patterns in



**Fig. 1.** Model development.

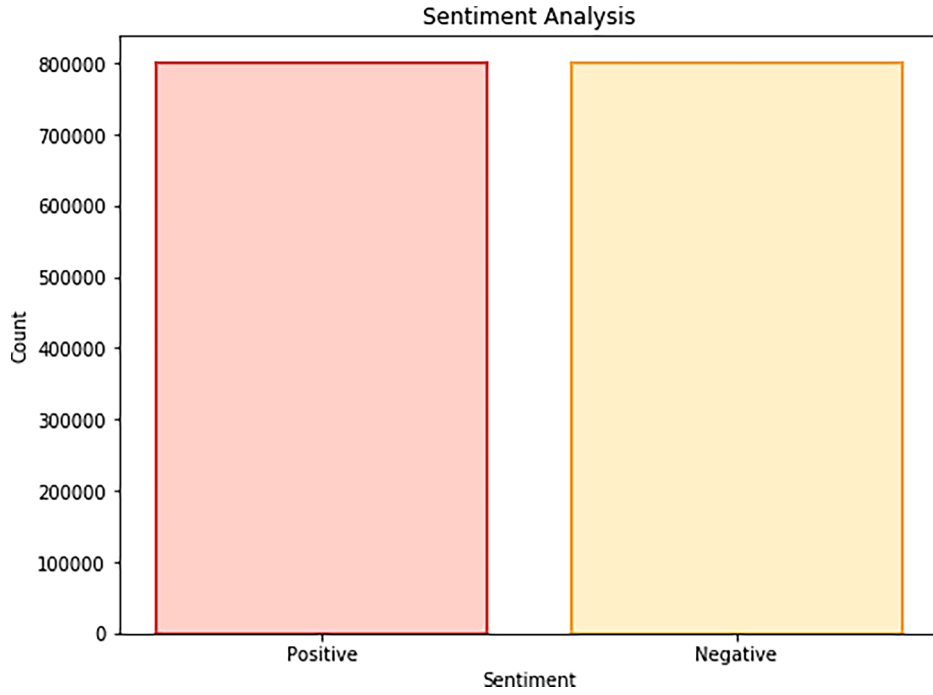


Fig. 2. Sentiment140.

**Table 2**  
Bi-LSTM model summary.

| Layer              | Param |
|--------------------|-------|
| Embedding          | 64    |
| Bidirectional LSTM | 256   |

the input data.

The Bi-LSTM model consists of an Embedding layer, a Bidirectional LSTM layer, and a Dense layer. The steps and transformations can be described mathematically as follows:

- **Embedding Layer:** The Embedding layer transforms the input sequences of tokens into dense vectors of fixed size. If the input sequence is  $x = (x_1, x_2, \dots, x_T)$  where  $T$  is the sequence length, and each  $x_t$  the embedding operation is:

$$e_t = W_e x_t$$

where  $W_e$  is the embedding matrix of size  $|V| \times d$  with  $|V|$  being the vocabulary size and  $d$  the embedding dimension (64 in this case).

- **Bidirectional LSTM Layer:** The Bidirectional LSTM processes the input sequences in both forward and backward directions. For each time step  $t$ , the forward LSTM computes:

$$\vec{h}_t = \overrightarrow{LSTM}(e_t, \vec{h}_{t-1})$$

Similarly, the backward LSTM computes:

$$\overleftarrow{h}_t = \overleftarrow{LSTM}(e_t, \overleftarrow{h}_{t+1})$$

The combined hidden state at each time step is:

$$h_t = [\vec{h}_t; \overleftarrow{h}_t]$$

where  $[\cdot]$  denotes the concatenation operation. Since each LSTM has 256 units, the combined hidden state  $h_t$  will have 512 units

- **Dense Layer:** The Dense layer takes the combined hidden state from the final time step and produces a single output:

$$\hat{y} = \sigma(W_d h_t + b_d)$$

where  $W_d$  is the weight matrix of size  $512 \times 1$ ,  $b_d$  is the bias term, and  $\sigma$  is the sigmoid activation function.

- **Model Parameters:** The model's trainable parameters include:
  - Embedding matrix  $W_e$  - LSTM parameters for both forward and backward LSTMs
  - Dense layer weights  $W_d$  and bias  $b_d$

These parameters allow the model to learn complex patterns from the input data.

## II. CNN Model Architecture

The neural network architecture in Table 3 includes an Embedding layer of 64 units, a 1D Convolutional layer of 256 units, a GlobalMaxPooling1D layer of 0.5 and two dense layer of 128 units with relu activation function and 1 unit with sigmoid activation function, respectively. The Embedding layer converts input sequences into dense vectors, the Conv1D layer applies filters for feature extraction, and GlobalMaxPooling1D extracts important features, followed by dense layers for classification.

**Table 3**  
CNN model summary.

| Layer              | Param |
|--------------------|-------|
| Embedding          | 64    |
| Conv1D             | 256   |
| GlobalMaxPooling1D | 0.5   |

- **Embedding Layer:** The Embedding layer transforms the input sequences of tokens into dense vectors of fixed size. If the input sequence is  $x = (x_1, x_2, \dots, x_T)$  where  $T$  is the sequence length, and each  $x_t$  the embedding operation is:

$$e_t = W_e x_t$$

where  $W_e$  is the embedding matrix of size  $|V| \times d$  with  $|V|$  being the vocabulary size and  $d$  the embedding dimension (64 in this case).

- **Convolutional Layer:** The 1D Convolutional layer applies filters of size 5 for feature extraction:

$$h_i = f(W_c \cdot e_{i:i+5} + b_c)$$

where  $W_c$  is the filter weight matrix,  $b_c$  is the bias term, and  $f$  is the ReLU activation function.

- **GlobalMaxPooling1D Layer:** The GlobalMaxPooling1D layer extracts the most important features:

$$h_{pool} = \max(h_1, h_2, \dots, h_n)$$

- **Dense Layers:** It consist of:

$$h_1 = \text{ReLU}(W_1 h_{pool} + b_1)$$

$$\hat{y} = \sigma(W_d h_t + b_d)$$

where  $W_1, W_2$  are the weight matrices,  $b_1, b_2$  are the bias terms, and  $\sigma$  is the sigmoid activation function.

- **Model Parameters:** The model's trainable parameters include:
  - Embedding matrix  $W_e$ - Convolutional layer filter weights  $W_c$  and bias  $b_c$
  - Dense layer weights  $W_1, W_2$  and bias  $b_1, b_2$

These parameters allow the model to learn complex patterns from the input data.

### III. GRU Model Architecture

The neural network architecture in Table 4 includes an Embedding layer of size 64 units, a Bidirectional GRU layer of 128 units, and two dense layers of 64 units with relu activation function and 1 unit of sigmoid activation function, respectively. The Embedding layer converts input sequences into dense vectors, while the Bidirectional GRU layer processes input sequences bidirectionally for better context understanding. The subsequent dense layers are used for feature extraction and prediction tasks.

- **Embedding Layer:** The Embedding layer transforms the input sequences of tokens into dense vectors of fixed size. If the input

sequence is  $x = (x_1, x_2, \dots, x_T)$  where  $T$  is the sequence length, and each  $x_t$  the embedding operation is:

$$e_t = W_e x_t$$

where  $W_e$  is the embedding matrix of size  $|V| \times d$  with  $|V|$  being the vocabulary size and  $d$  the embedding dimension (64 in this case).

- **Bidirectional GRU Layer:** The Bidirectional GRU processes the input sequences in both forward and backward directions. For each time step  $t$ , the forward GRU computes:

$$\vec{h}_t = \overrightarrow{GRU}(e_t, \vec{h}_{t-1})$$

Similarly, the backward GRU computes:

$$\overleftarrow{h}_t = \overleftarrow{GRU}(e_t, \overleftarrow{h}_{t+1})$$

The combined hidden state at each time step is:

$$h_t = [\vec{h}_t; \overleftarrow{h}_t]$$

where  $[\cdot]$  denotes the concatenation operation. Since each GRU has 128 units, the combined hidden state  $h_t$  will have 256 units

- **Dense Layers:** It consist of:

$$h_1 = \text{ReLU}(W_1 h_{pool} + b_1)$$

$$\hat{y} = \sigma(W_d h_t + b_d)$$

where  $W_1, W_2$  are the weight matrices,  $b_1, b_2$  are the bias terms, and  $\sigma$  is the sigmoid activation function.

### IV. RNN Model Architecture

The provided neural network architecture in Table 5 consists of an Embedding layer followed by a generic RNN layer and two Dense layers. The Embedding layer converts input sequences into dense vectors of size 64 for each token in a sequence length of 100. The RNN layer processes the embedded sequences and produces an output shape of 256, capturing sequential information within the data. Subsequently, two dense layers with 128 units with RELU activation function and 1 units with sigmoid activation function, respectively. The model has trainable parameters, making it suitable for prediction tasks.

- **Embedding Layer:** The Embedding layer transforms the input sequences of tokens into dense vectors of fixed size. If the input sequence is  $x = (x_1, x_2, \dots, x_T)$  where  $T$  is the sequence length, and each  $x_t$  the embedding operation is:

$$e_t = W_e x_t$$

**Table 4**  
GRU model summary.

| Layer             | Param |
|-------------------|-------|
| Embedding         | 64    |
| Bidirectional GRU | 128   |
| Dense             | 64    |

**Table 5**  
RNN model summary.

| Layer     | Param |
|-----------|-------|
| Embedding | 64    |
| RNN       | 256   |



where  $\mathbf{W}_e$  is the embedding matrix of size  $|V| \times d$  with  $|V|$  being the vocabulary size and  $d$  the embedding dimension (64 in this case).

- **Simple RNN Layer:** The Simple RNN processes the input sequences sequentially and produces an output at each time step. For each time step  $t$ , the output  $\mathbf{h}_t$  is computed as:

$$\mathbf{h}_t = \text{RNN}\left(\mathbf{e}_t, \vec{\mathbf{h}}_{t-1}\right)$$

where  $\mathbf{h}_{t-1}$  is the hidden state from the previous time step.

- **Dense Layers:** It consist of:

$$\mathbf{h}_1 = \text{ReLU}(\mathbf{W}_1 \mathbf{h}_{\text{pool}} + \mathbf{b}_1)$$

$$\hat{\mathbf{y}} = \sigma(\mathbf{W}_2 \mathbf{h}_1 + \mathbf{b}_2)$$

where  $\mathbf{W}_1$ ,  $\mathbf{W}_2$  are the weight matrices,  $\mathbf{b}_1$ ,  $\mathbf{b}_2$  are the bias terms, and  $\sigma$  is the sigmoid activation function.

### 3.6. Performance metrics

Sentiment analysis algorithms on Twitter are evaluated for both positive and negative attitudes. Here are the key performance metrics used:

#### 1) Accuracy

Accuracy measures the ratio of correctly predicted instances (true positives and true negatives) to the total instances:

$$\text{Accuracy} = \frac{\text{True Positives} + \text{True Negatives}}{\text{Total Instances}}$$

#### 2) Precision

Precision measures the proportion of true positive predictions among all positive predictions:

$$\text{Precision} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}}$$

#### 3) Recall

Recall measures the proportion of true positive predictions among all actual positive instances:

$$\text{Recall} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}}$$

#### 4) F1-score

F1-score is the harmonic mean of precision and recall, providing a balanced measure of a model's performance:

$$\text{F1 Score} = \frac{2 * \text{Precision} * \text{Recall}}{\text{Precision} + \text{Recall}}$$

### 3.7. Additional analysis

#### 1) Resource Monitoring

In both of these phases, we must look at the memory and CPU/GPU resources that are being consumed. Therefore, this system will see the computation needs of deep learning models and will be able to identify possible issues with scalability and efficiency as well. Resource utilization metrics are closely monitored to make the right decisions of optimizing the model's performance and the hardware allocation.

#### 2) Time Complexity Analysis

Accordingly, time complexity metrics learnt in training and testing were also evaluated in all models of deep learning. This evaluation induced the development in understanding the computational efficiency of the models. The factor of time complexity is considered as a constraint that may make scaling as well as optimizing process for better performance very difficult and sometimes impossible. Likewise, this extensive research compounded enough understanding to create models for more scaled and complex jobs. Identifying time complexity ( $O(f(n))$ ) during training and testing constitutes as a determinant of the computational efficiency as well as bottlenecks.

#### 3) Performance Comparison

Architectures and computing platforms are compared in DL models with regards to performance metrics, time-complexity, resource utilization, and scalability. Hence, the more suitable model for sentiment analysis tasks can be found via this method. This knowledge can be used by both practitioners and researchers in comparison of the different DL architectures for similar projects related to NLP.

## 4. Results

### 4.1. Comparison between Google Colab and PC

The study conducted by [27], focusing on sentiment analysis, utilized Twitter data [7], picked from Kaggle platform. It has 100,000 tweets that are partitioned into two sets of train and test. Text pre-processing methods are utilized as well LLM GPT 3.5 model. For carrying out sentiment analysis, we have developed a neural network model that uses Bi-LSTM, CNN, GRU and RNNs approaches. The performance was assessed by integrating with LLM and deep learning strategies.

The Bi-LSTM model in Table 6, shows consistent performance in terms of accuracy across both Google Colab and PC environments, achieving approximately 93 % accuracy on the training set and 73 % accuracy on the test set. This indicates that the model learns well and generalizes decently to unseen data. Table 7, when comparing the resource utilization, we observe significant differences between the platforms. Google Colab, with its access to powerful GPU resources, demonstrates efficient utilization with about 70.5 % CPU usage after training, while the PC sees a higher CPU utilization of around 98.4 %. The memory usage is also notable, with Google Colab initially allocating

**Table 6**  
Model performance comparison of PC and Google Colab.

| Model                  | Train accuracy | Test accuracy | Precision | Recall | F1 Score |
|------------------------|----------------|---------------|-----------|--------|----------|
| Google Colab (Bi-LSTM) | 93.45          | 73.00         | 0.74      | 0.73   | 0.73     |
| Google Colab (CNN)     | 99.88          | 73.00         | 0.74      | 0.73   | 0.73     |
| Google Colab (GRU)     | 93.95          | 73.00         | 0.73      | 0.73   | 0.73     |
| Google Colab (RNN)     | 51.98          | 52.00         | 0.51      | 0.77   | 0.61     |
| PC (Bi-LSTM)           | 93.84          | 73.00         | 0.74      | 0.71   | 0.73     |
| PC (CNN)               | 99.87          | 73.00         | 0.74      | 0.71   | 0.73     |
| PC (GRU)               | 95.40          | 73.00         | 0.73      | 0.73   | 0.73     |
| PC (RNN)               | 49.91          | 50.00         | 0.25      | 0.50   | 0.67     |

**Table 7**  
Time complexity & resource monitoring of PC and Google Colab.

| Model                  | Train time (s) | Test time (s) | Initial memory (GB) | Final memory (GB) | CPU usage before (%) | CPU usage after (%) |
|------------------------|----------------|---------------|---------------------|-------------------|----------------------|---------------------|
| Google Colab (Bi-LSTM) | 747.7          | 47.99         | 5.30                | 5.31              | 33.20 %              | 70.50 %             |
| Google Colab (CNN)     | 2.17           | 48.86         | 7.98                | 7.98              | 100.0 %              | 72.80 %             |
| Google Colab (GRU)     | 55.85          | 81.98         | 8.50                | 8.52              | 53.20 %              | 50.60 %             |
| Google Colab (RNN)     | 21.58          | 41.03         | 244.62              | 244.61            | 95.9 %               | 56.20 %             |
| PC (Bi-LSTM)           | 7252.82        | 102.47        | 0.0024              | 0.0024            | 78.40 %              | 98.40 %             |
| PC (CNN)               | 7.03           | 103.62        | 6.31                | 6.29              | 0.0 %                | 99.7 %              |
| PC (GRU)               | 68.11          | 68.11         | 6.88                | 6.91              | 18.4 %               | 99.0 %              |
| PC (RNN)               | 34.42          | 33.48         | 6.72                | 6.76              | 0.0 %                | 98.0 %              |

around 5.30 GB of memory, which reduces slightly after training, whereas the PC shows a more stable memory usage trend of 0.0024 GB. The training and test times on Google Colab are notably lower compared to the PC, with the Bi-LSTM model taking around 747 s to train on Google Colab and significantly longer, around 7252 s, on the PC.

The CNN model in Table 6, exhibits excellent performance in terms of accuracy, achieving close to 99 % on the training set and maintaining 73 % accuracy on the test set across both platforms. Despite the high accuracy, the precision, recall, and F1 scores remain consistent, showcasing the model's robustness. However in Table 7, the CNN model demands substantial computational resources, especially in terms of memory and CPU utilization. Google Colab's CPU utilization reaches 100 % during training, while initial CPU resource utilization is 0.0 % for the Personal PC. Memory-wise, both platforms show high initial memory allocations, with Google Colab allocating over 7.98 GB and the PC allocating around 6.31 GB. The CPU usage on Google Colab remains around 72.8 % after training, while the PC's CPU usage is 99.7 % during training due to GPU acceleration. Training times are significantly faster on Google Colab, taking only a few seconds compared to several minutes on the PC.

The GRU model in Table 6, demonstrates good accuracy and performance, similar to the Bi-LSTM model, with accuracy scores around 93–95 % on the training set and 73 % on the test set across the platforms. The precision, recall, and F1 scores are consistent, indicating stable model behavior. Table 7, resource utilization on Google Colab and PC for the GRU model shows similarities to the Bi-LSTM model, with CPU/GPU and memory usage being notable. Google Colab's CPU utilization is efficient, maintaining around 50–53 % after training with GPU acceleration, while the PC's CPU usage is at 99 %. Memory allocations and CPU usage follow similar trends as observed in previous models 8.5 GB, with Google Colab demonstrating slightly higher memory usage initially and 6.91 GB for PC. Training times for the GRU model are longer compared to the Bi-LSTM model, especially on the PC, where training times exceed 60 s.

The RNN model in Table 6, exhibits the lowest accuracy among the models, with training accuracy around 50–52 % and test accuracy at 50 % across both platforms. This indicates that the RNN model struggles to learn complex patterns in the data and may suffer from vanishing gradient issues. Precision, recall, and F1 scores also reflect the lower performance compared to other models. Table 7, resource utilization for the RNN model shows similarities to the LSTM and GRU models, with

notable differences in training and test times. Google Colab's faster processing capabilities reduce training times significantly compared to the PC, where training times are considerably higher. Utilization of memory and CPU/GPU follow trends of 244.62 GB for Google Colab and 6.76 GB for PC similar to other models, with higher initial memory allocations on Google Colab.

In general, the speed of training deep learning models on Google Colab and on a PC differs in the time scale, resource expenditure, and the efficiency of the compute. Google Colab makes it possible to use the GPUs for a quicker training time, effective use of the GPU and moderate dissipation of the resources during training. Nevertheless, Google Colab has some boundaries including, for instance, session duration or performance, that could result in shortening the long-term tasks.

In addition to this, a PC with GPU acceleration can effectively train deep learning models although it consumes a lot of resources in both CPU and GPU use and for the same job it takes long time to execute. Memory Management, e.g., is among the factors of the biggest importance, especially for the large models as the CNNs with the platforms demonstrate the huge preliminary memory allocation margins.

Choosing Google Colab or a PC depends on various factors, such as project requirements, budget constraints, dataset size and continuous access to resources among others. For example, for short-term projects and those that consume too many resources, Google Colab is convenient and cost-effective. On the other hand, long-term projects require specific hardware configurations or custom built PCs with optimized hardware configurations to provide better control and flexibility. Balanced computing power, memory usage management and training times are paramount in developing efficient deep learning models.

Deeper into comparison and analysis including the overall total available GPU's, CPUs' and RAM on both Google Colab and PC with how much was used during model training.

#### 1) Google Colab

##### - Total Resources:

- GPU: Tesla K80, Tesla P100, Tesla T4, or Tesla P4 GPUs.
- **Tesla K80**
  - GPU Memory: 24 GB (12 GB per GPU, dual GPU configuration).
  - Memory Bandwidth: 480 GB/s.
  - Boost Clock Speed: 875 MHz.
- **Tesla P100**
  - GPU Memory: 16 GB HBM2.
  - Memory Bandwidth: 732 GB/s.
  - Boost Clock Speed: 1328 MHz.
- **Tesla T4**
  - GPU Memory: 16 GB GDDR6.
  - Memory Bandwidth: 320 GB/s.
  - Boost Clock Speed: 1590 MHz.
- CPU: Multi-core CPUs with a total of 40 CPU cores.
- Google Colab typically provides access to high- core-count CPUs like Intel Xeon processors.
- Base Clock Speed: Around 2.3 GHz to 2.7 GHz. Cache: L1, L2, and L3 cache (varies with CPU model).
- **RAM:** Total Memory: 376.58 GB . RAM Type: DDR4.
- Speed: Around 2400 MHz to 2933 MHz.
- Provides excellent support for large dataset processing, enabling larger deep learning models to fit into memory without requiring paging or data swapping.

##### - Resource Utilization:

- GPU: High utilization, often close to 100
- CPU: Fluctuates during training but stabilizes afterward.
- RAM: High initial allocation but efficiently managed.

#### 2) PC

##### - Total Resources:

- GPU: NVIDIA GeForce or AMD Radeon GPUs (Total GPUs: 1).

**NVIDIA GeForce RTX 3060 (example mid- range GPU):**

- GPU Memory: 12 GB GDDR6.
- Memory Bandwidth: 360 GB/s.
- Clock Speed: 1777 MHz.

#### AMD Radeon RX 5700 XT (example AMD GPU):

- GPU Memory: 8 GB GDDR6.
- Memory Bandwidth: 448 GB/s.
- Clock Speed: 1905 MHz.

#### CPU: Multi-core CPUs with a total of 4 CPU cores.

- 4-core CPU (example: Intel Core i5-9600 K):
- Clock Speed: Up to 4.6 GHz.
- Cache: 9 MB Intel Smart Cache.

#### RAM: Total RAM capacity of 7.86 GB

- RAM Type: DDR4.
- Speed: Typically 2400 MHz to 3200 MHz depending on the configuration.
- Limited memory can restrict the size of datasets or models that can be loaded into memory at once.

#### - Resource Utilization:

- GPU: Heavily utilized during training.
- CPU: Usage depends on GPU acceleration and parallel processing.
- RAM: Similar to Google Colab, substantial initial allocation.

### 3) Comparison and Analysis

- GPU Comparison: Colab (Tesla GPUs) provides high-end GPUs such as Tesla P100 and T4, which are designed for data centers and deep learning workloads. These GPUs have significantly higher memory (up to 24 GB in the K80) than typical consumer-grade GPUs, though their clock speed may be higher. The specialized nature of Tesla GPUs, especially with their optimized memory bandwidth and tensor cores (in T4), gives them an advantage in large-scale training tasks.
- CPU Comparison: Google Colab's CPUs have up to 40 cores, which are typically high-end Intel Xeon processors, offering high parallelism ideal for managing the overheads of large machine-learning models. By contrast, most PCs would have a quad-core CPU like the Intel i5-9600 K or AMD Ryzen 5 3600, both of which are much less capable in terms of parallel processing but have higher individual clock speeds, which can be more beneficial for single-threaded tasks.
- RAM Comparison: Google Colab's 376.58 GB RAM vastly outclasses a typical PC's 8 GB RAM (or similar), especially in handling large datasets, deep learning models, or memory-intensive operations. With less RAM on a PC, the system could run into memory bottlenecks when trying to process larger models or datasets, whereas Colab can handle these tasks much more efficiently.

Therefore, Google Colab, being the Cloud-computing platform, facilitates potential researchers by availing the efficient resource management of powerful cloud-based resources, though it is not extensively flexible enough and does not present the option of controlling hardware configuration which may be done with PCs with GPU acceleration. The best way to speed up the deep learning process is by recognizing idiosyncrasies of the resource usage which will thus assist in the decision of choosing the suitable platform for project requirements and budget. It, then, implies the significance of balancing computational resources in terms of memory usage and training efficiency in developing and deploying refined and useful deep learning methods.

### 4.2. Comparison of Utkarsh server vs. Ohio supercomputer

The experiment [27] was initially conducted on a PC and Google Colab using a dataset of 100,000 Twitter tweets due to memory and resource restrictions. Subsequently, the experiment was upgraded to supercomputing machines and using 1.6 million records on the Ohio Supercomputer and the Utkarsh server. The performance comparison of deep learning models between Ohio Supercomputer and the Utkarsh environment unveils intriguing insights into their efficiency and resource utilization. Across various models such as Bi-LSTM, CNN, GRU, and RNN, both environments exhibit notable differences in accuracy, resource consumption, and training times.

For instance, the Bi-LSTM model in Table 8 achieves a consistent accuracy of around 92.48 % and 91.77 % on both training and test sets in both environments. However, Ohio Supercomputer showcases moderate resource utilization with GPU, CPU, and memory usage below 5 %, while the Utkarsh environment demonstrates higher CPU and memory utilization percentages, indicating a heavier computational load during training tabulated in Table 9. The training times differ significantly, with Ohio Supercomputer completing training in approximately 100 s, whereas the Utkarsh environment requires significantly longer, exceeding 30,000 s. Similarly, the CNN model in Table 8 demonstrates excellent accuracy, reaching around 95 % on the training set and maintaining 74–75 % accuracy on the test set across both platforms. However, resource utilization reflects the computational demands of CNN architectures, with Ohio Supercomputer showing balanced resource utilization and Utkarsh displaying higher CPU and memory utilization during training tabulated in Table 9. Despite these differences, training and testing times remain relatively low for both platforms, around 30–40 s.

The GRU model in Table 8 performs consistently well on both platforms, with accuracy scores ranging from 92.84 to 92.63 % on the training set and 75 % on the test set. Resource utilization patterns are similar to other models, with Ohio Supercomputer exhibiting slightly lower utilization percentages compared to Utkarsh tabulated in Table 9. Training and testing times for the GRU model are moderate, ranging from 70 to 80 s.

In contrast, the RNN model in Table 8 demonstrates mixed performance across the platforms, with accuracy ranging from 50.87 % to 51.98 % on the training set and 50–52 % accuracy on the test set. Resource utilization and time complexity vary significantly, with Ohio Supercomputer showing lower CPU and memory utilization percentages during training, while the Utkarsh environment exhibits higher utilization percentages, especially for memory. Training times on Ohio Supercomputer are relatively low, around 28–29 s, while Utkarsh environment experiences longer training times exceeding 800 s tabulated in Table 9.

In conclusion, the choice between Ohio Supercomputer and the Utkarsh environment depends on factors such as dataset size, model complexity, budget constraints, and required training times. Ohio

**Table 8**

Model performance comparison of Utkarsh server and Ohio supercomputer.

| Model                        | Train accuracy | Test accuracy | Precision | Recall | F1 Score |
|------------------------------|----------------|---------------|-----------|--------|----------|
| Ohio Supercomputer (Bi-LSTM) | 92.48          | 75.00         | 0.75      | 0.76   | 0.75     |
| Ohio Supercomputer (CNN)     | 95.39          | 74.00         | 0.75      | 0.74   | 0.74     |
| Ohio Supercomputer (GRU)     | 92.84          | 75.00         | 0.75      | 0.75   | 0.75     |
| Ohio Supercomputer (RNN)     | 51.98          | 52.00         | 0.51      | 0.77   | 0.61     |
| Utkarsh (Bi-LSTM)            | 91.77          | 75.00         | 0.76      | 0.74   | 0.75     |
| Utkarsh (CNN)                | 95.36          | 75.00         | 0.73      | 0.78   | 0.75     |
| Utkarsh (GRU)                | 92.63          | 75.00         | 0.75      | 0.74   | 0.75     |
| Utkarsh (RNN)                | 50.87          | 50.00         | 0.51      | 1.00   | 0.67     |



**Table 9**

Time complexity & resource monitoring of Utkarsh server and Ohio supercomputer.

| Model                        | Train time (s) | Test time (s) | Initial memory (GB) | Final memory (GB) | CPU usage before (%) | CPU usage after (%) |
|------------------------------|----------------|---------------|---------------------|-------------------|----------------------|---------------------|
| Ohio Supercomputer (Bi-LSTM) | 100.82         | 100.82        | 0.052               | 0.052             | 3.40 %               | 2.40 %              |
| Ohio Supercomputer (CNN)     | 38.76          | 38.76         | 115.17              | 114.95            | 69.30 %              | 63.20 %             |
| Ohio Supercomputer (GRU)     | 77.58          | 77.44         | 120.78              | 120.79            | 4.00 %               | 2.50 %              |
| Ohio Supercomputer (RNN)     | 28.67          | 24.17         | 8.83                | 8.85              | 85.30 %              | 51.90 %             |
| Utkarsh (Bi-LSTM)            | 31,920.4       | 1016.4        | 0.0525              | 0.0526            | 73.20 %              | 16.50 %             |
| Utkarsh (CNN)                | 32.73          | 38.14         | 248.84              | 248.94            | 67.00 %              | 80.00 %             |
| Utkarsh (GRU)                | 804.15         | 742.42        | 300.97              | 300.98            | 74.70 %              | 74.50 %             |
| Utkarsh (RNN)                | 21.58          | 41.03         | 248.90              | 248.88            | 85.90 %              | 100.00 %            |

Supercomputer offers advantages in terms of faster training times and efficient resource utilization, making it suitable for large-scale deep learning tasks. However, Utkarsh can also handle deep learning tasks effectively with optimized resource allocation strategies. Balancing computational resources, memory management, and training efficiency is crucial for successful deep learning model development and deployment across diverse computing platforms. The comparison between the Ohio Supercomputer and the Utkarsh environment in terms of available computational resources and their utilization during model training provides valuable insights into their efficiency.

#### 1) Ohio Supercomputer

–Total Resources:

- GPU: Ohio Supercomputer has 1 GPU available for computation tasks.
- CPU: The system is equipped with 40 CPU cores, providing substantial parallel processing capabilities.
- RAM: A massive amount of RAM is available, totaling approximately 376.53 GB.

–Resource Utilization:

GPU Utilization: During training and testing, the GPU utilization remains relatively low, ranging from 2.40 % to 4.00 %, indicating efficient utilization of GPU resources without overloading the system.

CPU Utilization: CPU cores are moderately utilized, with percentages ranging from 3.40 % to 4.00 %, efficiently distributing the workload across multiple cores during training.

RAM Utilization: RAM usage is within reasonable limits, ranging from 2.40 % to 4.00 %, effectively managing memory allocation during model training.

#### 2) Utkarsh Environment

–Total Resources:

GPU: The availability of GPUs varies across different experiments. In some cases, there is no GPU available, while in others, there are 1 or 2 GPUs present.

CPU: Similar to the Ohio Supercomputer, Utkarsh also has 40 CPU cores available for computations.

RAM: The RAM capacity matches that of the Ohio Supercomputer, totaling approximately 376.58 GB.

–Resource Utilization:

- GPU Utilization: When GPUs are available, their utilization ranges from 63.20 % to 80.00 %, indicating a significant workload for the GPUs during model training and testing.
- CPU Utilization: CPU cores are heavily utilized, especially in experiments where GPUs are absent, with CPU utilization ranging from 16.50 % to 100 %, showcasing varied computational loads across experiments.

- RAM Utilization: RAM usage percentages range from 2.50 % to 74.70 %, depending on the experiment, with higher RAM utilization observed in experiments with intensive computational tasks or larger model sizes.

## 5. Discussion

Google Colab, being the Cloud-computing platform, facilitates potential researchers by availing the efficient resource management of powerful cloud-based resources, though it is not extensively flexible enough and does not present the option of controlling hardware configuration which may be done with PCs with GPU acceleration. The best way to speed up the deep learning process is by recognizing idiosyncrasies of the resource usage which will thus assist in the decision of choosing the suitable platform for project requirements and budget. It, then, implies the significance of balancing computational resources in terms of memory usage and training efficiency in developing and deploying refined and useful deep learning methods.

Ohio Supercomputer demonstrates lower GPU utilization compared to Utkarsh, indicating a more distributed workload across different hardware setups. Both environments effectively utilize CPU cores, but Utkarsh shows higher CPU usage in experiments without GPU acceleration. RAM utilization is generally efficient in both environments, with variations based on the computational complexity of the tasks.

Impact on Performance: Higher resource utilization, especially on GPUs, may lead to faster training times but can also increase costs and system heating. Balanced resource usage is crucial for optimal performance and system stability. In summary, understanding and optimizing resource utilization play a vital role in deep learning experiments. Balancing computational demands with available resources ensures efficient model training, optimal performance, and cost-effectiveness in both research and production environments.

## 6. Conclusion

In conclusion, our study delves into various deep learning techniques, including Bi-LSTM, CNN, GRU, and RNN, applied to sentiment analysis tasks. Leveraging Twitter datasets and GPT-3.5 models, we conducted extensive experiments using different computing platforms, such as Google Colab, PCs, Ohio Supercomputer, and the Utkarsh server. Our evaluation criteria encompassed training and test accuracies, precision, recall, F1 scores, as well as resource tracking metrics like memory usage, time complexity, and CPU/GPU utilization. The comparison between these platforms revealed intriguing insights into their efficiency, assisting model deployment decisions across platforms optimized for performance and resource utilization. While higher resource utilization, particularly on GPUs, may lead to faster training times, it also incurs increased costs and system heating. Thus, striking a balance in resource usage is crucial for optimal performance and system stability. Looking ahead, exploring advanced methodologies like Explainable AI on next-generation AI systems, particularly concerning collaborative learning over distributed environments such as servers tailored for largescale data processing, remains essential. In summary, our research underscores the significance of understanding and optimizing resource

utilization in deep learning experiments. By balancing computational demands with available resources, we can achieve efficient model training, optimal performance, and cost-effectiveness in both research and production environments.

### CRedit authorship contribution statement

**Lavanya B N:** Writing – review & editing, Writing – original draft, Validation, Methodology, Formal analysis. **Anitha Rathnam K V:** Methodology. **Abhishek Appaji:** Writing – review & editing, Supervision, Software, Resources, Data curation, Conceptualization. **Kiran K:** Validation, Supervision. **P. Deepa Shenoy:** Supervision, Project administration. **Venugopal K R:** Supervision.

### Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

### Acknowledgment

We extend our gratitude to Dr. Lei Wang, Ph.D., Professor in Psychiatry and Behavioral Health and Neuroscience at Ohio State University Wexner Medical Center, USA, for providing access to the Ohio Supercomputer Center, enabling further stimulation.

The authors would also like to acknowledge government of India and CDAC for allowing us to use PARAM Utkarsh, a High Performance Computing System setup at C-DAC, Bangalore, India under the National Supercomputing Mission (NSM), Government of India.

### Data availability

The data is available at the URL <https://www.kaggle.com/datasets/kazanov/sentiment140>.

### References

- [1] W. He, S. Zha, L. Li, Social media competitive analysis and text mining: A case study in the pizza industry, *International Journal of Information Management* 33 (3) (2013) 464–472, <https://doi.org/10.1016/j.ijinfomgt.2013.04.001>.
- [2] G. Vinodhini, R.M. Chandrasekaran, Sentiment analysis and opinion mining: a survey, *International Journal* 2 (6) (2012) 282–292.
- [3] E. Rosenberg, C. Tarazona, F. Mallor, Sentiment analysis on Twitter data towards climate action, *Results in Engineering* 19 (2023) 101287, <https://doi.org/10.1016/j.rineng.2023.101287>.
- [4] M. Wankhade, A.C.S. Rao, C. Kulkarni, A survey on sentiment analysis methods, applications, and challenges, *Artificial Intelligence Review* 55 (7) (2022) 5731–5780, <https://doi.org/10.1007/s10479-021-02778-2>.
- [5] O. Habimana, Y. Li, R. Li, X. Gu, G. Yu, Sentiment analysis using deep learning approaches: an overview, *Science China Information Sciences* 63 (2020) 1–36, <https://doi.org/10.1007/s11438-019-1949-z>.
- [6] B.N. L, R.K.V A, K. K, A. Appaji, P.D. Shenoy, K.R. V, Sentiment analysis of textual data using word embedding and deep learning approaches, in: 2023 IEEE North Karnataka Subsection Flagship International Conference (NKCon), 2023, pp. 1–6, <https://doi.org/10.1109/NKCon59507.2023.10396439>.
- [7] M.M. Kazanova, “Sentiment140 Dataset,” 2010. Available: <https://www.kaggle.com/datasets/kazanov/sentiment140>.
- [8] K.L. Tan, C.P. Lee, K.M. Lim, K.S.M. Anbananthen, Sentiment analysis with ensemble hybrid deep learning model, *IEEE Access* 10 (2022) 103694–103704, <https://doi.org/10.1109/ACCESS.2022.1200323>.
- [9] P.C. Shilpa, R. Shereen, S. Jacob, P. Vinod, Sentiment analysis using deep learning, in: 2021 Third International Conference on Intelligent Communication Technologies and Virtual Mobile Networks (ICICV), 2021, pp. 930–937, <https://doi.org/10.1109/ICICV52186.2021.9622225>.
- [10] L. Kurniasari, A. Setyanto, Sentiment analysis using recurrent neural network, *Journal of Physics: Conference Series* 1471 (1) (2020) 012018, <https://doi.org/10.1088/1742-6596/1471/1/012018>.
- [11] R.S. Chauhan, *Sentiment Analysis of Customer Reviews Using Deep Learning Techniques*, PhD Thesis, National College of Ireland, Dublin, 2022.
- [12] B.S. Ainapure, R.N. Pise, P. Reddy, B. Appasani, A. Srinivasulu, M.S. Khan, N. Bizon, Sentiment Analysis of COVID-19 Tweets Using Deep Learning and Lexicon-Based Approaches, *Sustainability* 15 (3) (2023) 2573, <https://doi.org/10.3390/su15032573>.
- [13] M.A. Aziz, M.M. Islam, M.S.P. Munshi, S. Samiha, M.M. Khan, Sentimental Analysis on reviews of Protein Supplement using RNN-LSTM, in: 2022 6th International Conference on Computing Methodologies and Communication (ICCMC), 2022, pp. 796–800, <https://doi.org/10.1109/ICCMC55342.2022.9799275>.
- [14] Y.E. Rizk, W.M. Asal, Sentiment analysis using machine learning and deep learning models on movies reviews, in: 2021 3rd Novel Intelligent and Leading Emerging Sciences Conference (NILES), 2021, pp. 129–132, <https://doi.org/10.1109/NILES53570.2021.9600223>.
- [15] K.K. Mohbey, Sentiment analysis for product rating using a deep learning approach, in: 2021 International Conference on Artificial Intelligence and Smart Systems (ICAIS), 2021, pp. 121–126, <https://doi.org/10.1109/ICAIS52400.2021.9484342>.
- [16] S. Sukheja, S. Chopra, M. Vijayalakshmi, Sentiment Analysis using Deep Learning—A survey, in: 2020 International Conference on Computer Science, Engineering and Applications (ICCSEA), 2020, pp. 1–4, <https://doi.org/10.1109/ICCSEA51792.2020.9301402>.
- [17] M.I. Prabha, G. Umarani Srikanth, Survey of Sentiment Analysis Using Deep Learning Techniques, in: 2019 1st International Conference on Innovations in Information and Communication Technology (ICICT), Chennai, India, 2019, pp. 1–9, <https://doi.org/10.1109/ICICT1.2019.8741438>.
- [18] T. Abdullah, A. Ahmet, Deep Learning in Sentiment Analysis: Recent Architectures, *ACM Computing Surveys* 55 (8) (August 2023) 159, <https://doi.org/10.1145/3548772>. Article.
- [19] V.K. Chandra Mula, L. Kumar, L.B. Murthy, A. Krishna, Software Sentiment Analysis using Deep-learning Approach with Word-Embedding Techniques, in: 2022 17th Conference on Computer Science and Intelligence Systems (FedCSIS), Sofia, Bulgaria, 2022, pp. 873–882, <https://doi.org/10.15439/2022F13>.
- [20] E. Dogan, B. Kaya, Deep learning based sentiment analysis and text summarization in social networks, in: 2019 International Artificial Intelligence and Data Processing Symposium (IDAP), 2019, pp. 1–6, <https://doi.org/10.1109/IDAP.2019.8818423>.
- [21] R. Pupale Shrivastava, P. Singh, Enhancing Aggression Detection using GPT-2 based Data Balancing Technique, in: 2021 5th International Conference on Intelligent Computing and Control Systems (ICICCS), 2021, pp. 1345–1350, <https://doi.org/10.1109/ICICCS52400.2021.9554437>.
- [22] T. Obinwanna, P. Brandtner, Enhancing Sentiment Analysis with GPT—A Comparison of Large Language Models and Traditional Machine Learning Techniques, in: A.K. Nagar, D.S. Jat, D. Mishra, A. Joshi (Eds.), *Intelligent Sustainable Systems*, Springer Nature, Singapore, 2024, pp. 187–197.
- [23] M. Leippold, Sentiment spin: Attacking financial sentiment with GPT-3, *Finance Research Letters* 55 (2023) 103957, <https://doi.org/10.1016/j.frl.2023.103957>.
- [24] L. Mathew, V.R. Bindu, A Review of Natural Language Processing Techniques for Sentiment Analysis using Pre-trained Models, in: 2020 Fourth International Conference on Computing Methodologies and Communication (ICCMC), 2020, pp. 340–345, <https://doi.org/10.1109/ICCMC50036.2020.9286883>.
- [25] D. CarneroPrado, L. Villa, E. Johnson, C.C. Dobrescu, A. Barragan, B. García-Martínez, Comparative Study of Large Language Models as Emotion and Sentiment Analysis Systems: A Case-Specific Analysis of GPT vs. IBM Watson, in: J. Bravo, G. Urzáiz (Eds.), *Proceedings of the 15th International Conference on Ubiquitous Computing & Ambient Intelligence (UCAmi 2023)*, Springer Nature Switzerland, 2023, pp. 229–239.
- [26] D.K. Behera, J. Hota, A.R. Swain, Ensemble Graph Neural Networks for Fake News Detection Using User Engagement and Text Features, *Results in Engineering* (2024) 103081, <https://doi.org/10.1016/j.rineng.2024.103081>.
- [27] B.N. L, R.K.V A, A. Appaji, K. K, P.D. Shenoy, K.R. V, Leveraging LLM GPT-3.5 for Sentiment Analysis: A Comparative Study Across PC and Google Colab, in: 2024 IEEE International Conference on Electronics, Computing and Communication Technologies (CONECT), Bangalore, India, 2024, pp. 1–7, <https://doi.org/10.1109/CONECT62155.2024.10677300>.